Steam Review Filter and Score

Capstone Project Documentation — Data Science & AI, Institute of Data

Author: Sean C. | Repo: github.com/brighthikaru/steam-review-filter-and-score | Live app: steam-review-filter-and-score.streamlit.app

AI assistance disclosure: the summarization component (model selection/testing across seven attempts, and the per-review-then-join method described under Model 3 in Section 8) and the Streamlit app (UI and deployment) were built with assistance from Claude (Anthropic), used as a coding assistant since I had no prior experience with generative summarization models or app deployment. The core data science work -- EDA (Section 7) and the Model 1/Model 2 comparisons in Section 8 -- is my own.

1. Problem Statement

A one-word Steam review — "good game," "bad," "10/10" — records a vote but explains nothing: no gameplay mechanics, no bugs, no pacing, nothing that helps another player judge whether the game is actually for them. These thin reviews sit alongside genuinely informative ones at roughly equal visibility in Steam's review list, and both count identically toward the aggregate score. That makes it harder for a prospective buyer to find the reviews that would actually inform a purchase decision, and harder for developers to separate substantive feedback from noise. Steam applies some internal filtering, but never surfaces the substantive reviews specifically, nor a "real sentiment" number. Left unaddressed, purchase decisions and developer feedback both rest on a review list where detail is buried rather than surfaced. No published prior work solving this specific problem for Steam was identified.

2. Industry / Domain

Digital game storefronts, and more broadly any platform built on user-generated reviews — app stores, e-commerce sites, anywhere reviews turn into a public score. The chain runs from player experience to review text to aggregate score to purchase decisions by future players, and reputation signal to developers. This is directly relevant to any UGC review platform facing the same low-effort-review dilution problem; the same two-model approach — quality filter plus text-based sentiment — would transfer with minimal changes.

3. Stakeholders

- Players deciding whether to buy a game — want real detail (gameplay, bugs, pacing, value) to judge fit, not just a vote count or a blended score.
- Developers and publishers — want genuine, substantive feedback surfaced clearly, especially after a patch, price change, or controversy, when low-effort review volume spikes.
- Steam itself — already filters some abuse; surfacing the substantive reviews specifically, plus a transparent text-only "real sentiment" score, is a natural extension its own review list doesn't currently provide.

4. Business Question

Can the substantive reviews for a game be surfaced and the low-effort/junk ones filtered out, using nothing but the review text, so a player gets real detail to inform a buying decision? A secondary, supporting question: does that same filtering produce a "real sentiment" score that reflects genuine community sentiment more accurately than the raw aggregate? To be useful in practice, the quality filter needs precision high enough that genuine, substantive reviews are rarely misclassified as junk, and the sentiment model needs to track a reviewer's real vote closely enough to work as a trustworthy proxy where a vote isn't available. Section 8 covers how both models measure up against that bar.

5. Data Question

Does review text alone carry a generalisable signal for (a) sentiment and (b) review quality, and does that signal hold across genuinely different games and genres — not just the ones a model was trained on?

6. Data

140,000 reviews across 7 games, 20,000 each, pulled live from Steam's public appreviews API (no key required, cursor-paginated). Games were chosen to span distinct genres so the results are not just an artefact of one community's writing style:

Game	Genre	Total English reviews	% sampled
Helldivers 2	Co-op action shooter	816,065	2.5%
Team Fortress 2	Multiplayer FPS	739,332	2.7%
Cyberpunk 2077	Open-world RPG	411,481	4.9%
Resident Evil Requiem	Survival horror	79,019	25.3%
Slay the Spire 2	Deck-building roguelike	72,061	27.8%
Forza Horizon 5	Racing	97,866	20.4%

Tekken 8	Fighting	43,200	46.3%

20,000 per game is a practical cap, not an exhaustive pull — it keeps every genre contributing equally to what the models learn and keeps training time manageable. On reliability: Steam's own language tag is reviewer-set, not content-verified (~1.2% of "English"-tagged reviews turned out to be in another language), so a second, content-based filter was added to drop any review containing non-Latin script. The project is scoped to English-only by design, since a review that cannot be manually verified is not defensible to include. The data is available on an ongoing basis via the same API for any future re-collection.

7. Data Analysis (EDA highlights)

- Review length is heavily right-skewed — a large share of reviews are only a handful of characters, exactly what the quality filter targets.
- Playtime at time of review has a median of ~48 hours; only 1.2% of reviews come from accounts with under 1 hour played, so the low-playtime signal is a strict, narrow flag, not a majority pattern.
- Two of seven games sit below 60% positive: Tekken 8 (real, ongoing monetisation backlash) and Helldivers 2. Helldivers 2 is notable because its all-time Steam score is "Very Positive" (~80%), but this sample of its most recent 20,000 English reviews shows only 51.6% — a useful reminder that a recent-reviews sample can diverge sharply from an all-time aggregate.

8. Modelling

Features: four structural signals — is_very_short, is_low_playtime, is_duplicate_text, is_new_reviewer — are only used to construct a proxy "low-effort" training label; neither deployed model ever sees them at inference time. That keeps both models usable on review text pulled from anywhere, with nothing per-game to recalibrate.

Feature selection: no ground-truth "low-effort" label exists, so one was engineered from length, playtime, and duplicate-text signals — chosen because they are game-agnostic and timing-independent. TF-IDF (unigrams + bigrams, vocabulary built only from the training split) supplies text features for the classical models; the CNN and LSTM learn their own embeddings.

Model 1 — Quality (low-effort) filter: six algorithms compared

Model	Precision	Recall	F1	ROC-AUC
Naive Bayes	0.790	0.604	0.685	0.911
Random Forest	0.705	0.971	0.817	0.954
Linear SVM	0.845	0.891	0.867	0.956
Logistic Regression	0.827	0.931	0.876	0.962
Stacking (LogReg+NB+RF)	0.867	0.900	0.883	0.963
CNN (deployed)	0.930	0.957	0.943	0.987

CNN selected: a clear, consistent win (ROC-AUC 0.987 vs. 0.963 for the best classical model), with precision holding between 0.90 and 0.96 across all seven genres rather than being driven by results from only a few games.

Model 2 — Sentiment scoring: four algorithms compared

Model	Precision	Recall	F1	ROC-AUC
LSTM	0.958	0.915	0.936	0.950
CNN	0.956	0.922	0.939	0.955
Logistic Regression	0.961	0.914	0.937	0.959
Stacking (deployed)	0.942	0.954	0.948	0.959

Stacking selected: it wins on F1 and recall, and on agreement with Steam's real vote (91.8% vs. Logistic Regression's 90.4%), though it ties Logistic Regression exactly on ROC-AUC (0.959 both). This is reported in full rather than omitting the metric that favours Logistic Regression. Deep learning did not clearly improve results for sentiment the way it did for the quality filter, a useful negative result.

Training cost: the quality CNN converges in ~3 epochs (~30 seconds on CPU); sentiment Stacking fits in ~25 seconds. Both trained locally on CPU with scikit-learn and TensorFlow/Keras — no cloud or GPU needed at this data volume.

Classification threshold

Both deployed models classify at the default 0.5 probability cutoff — quality_cnn.keras via score >= 0.5 in live_scoring.py, and the Stacking sentiment model via scikit-learn's default .predict() — rather than being tuned against a specific precision or recall target. This is a deliberate simplification, not an oversight: with no verified ground truth for "low-effort" (see Feature selection above), there is no labelled validation set to tune a business-driven threshold against, and 0.5 treats a false "flagged as junk" and a false "missed junk" as equally costly, which may not match how a stakeholder actually weighs the two. In practice this shows up as an

asymmetry worth noting: the quality filter's precision (0.90–0.96) consistently runs ahead of its recall, so on the rare misclassification it is more likely to flag a genuine review than to let a junk review through — the safer direction of error for a filter feeding a public-facing score, but worth stating explicitly rather than leaving implicit. Calibrating this threshold against an explicit target (e.g. "never flag more than 2% of genuine reviews") is listed as future work in Section 15.

Model 3 — Summarization (bonus, beyond the two required)

The app generates a plain-English "Players liked: ..." / "Players disliked: ..." summary from the kept reviews. This is a bonus feature, not a formally compared third model — there is no precision/recall/ROC-AUC to score a generated summary against, so unlike Models 1 and 2 above, what follows is a build log across seven attempts, not a metrics comparison:

Approach	Outcome	Why rejected / replaced
t5-small (generic)	Safe on memory	Copy-pasted raw fragments rather than condensing them
bart-large-cnn (~400M)	Coherent prose	OOM-crashed a live free-tier deployment on first request
distilbart-cnn-12-6	Never loaded	PyTorch-only weights, no TF-native option
flan-t5-base	Hallucinated	Invented a detail ("graphics") not in the source reviews
flan-t5-small	Vague, one-sided	Ignored negative reviews, produced generic praise
TF-IDF (extractive)	Safe, grounded	Couldn't produce real prose, phrase-list only
Falconsai/text_summarization (deployed)	Deployed	60.5M params / 242MB — same memory class as generic t5-small, but fine-tuned specifically for summarization

Why this one, with evidence, not just elimination: local memory profiling (test_summarizer_memory.py, run against real Helldivers 2 reviews) showed the model loading at ~764MB and peaking at ~1053MB resident memory — comfortably inside the free-tier host's budget that had already OOM-crashed bart-large-cnn on its first real request. Across three separate local test runs, generated output showed no fabricated detail traceable to any source review — the same check that had caught flan-t5-base inventing "graphics" out of nothing. Falconsai was reached for specifically, not by accident, because it is a t5-small-class checkpoint (matching the model already proven safe on memory) but fine-tuned for summarization rather than generic multi-task text-to-text — that combination is what the other six attempts didn't have.

One problem remained even after landing on Falconsai/text_summarization: summarizing several reviews in a single pass reliably dropped at least one review from the output entirely, the same failure mode that sank the original t5-small. The fix was method, not model — each kept review is summarized individually, then the short results are joined, guaranteeing every review is represented since none can be dropped once each gets its own generation pass.

9. Outcomes

The quality filter reliably separates thin, low-information reviews from substantive ones, which lets the app surface real detail to a prospective buyer instead of a raw, unsorted review list. As a secondary check on that filtering, the positive percentage drops for every one of the seven games once low-effort reviews are removed — a consistent, non-obvious finding, because low-effort reviews skew more positive than substantive ones. Tekken 8 crosses a full Steam category ("Mixed" → "Mostly Negative", 47.6% → 39.2%), and so does Forza Horizon 5 ("Very Positive" → "Mostly Positive", 88.7% → 79.7%). The sentiment model's text-only prediction agrees with the reviewer's real vote 82–96% of the time depending on the game, confirming it is reading sentiment correctly rather than guessing.

10. Implementation

The two required models are packaged as portable artifacts — the CNN quality filter as a single .keras file (its vocabulary is baked into the model, so no separate vectorizer is needed at inference time), and the Stacking sentiment model plus its TF-IDF vectorizer as joblib files. The summarizer downloads its own pretrained weights (Falconsai/text_summarization, 242MB) from Hugging Face on first load rather than using a locally-trained artifact. The Streamlit app loads all three models and, for any game someone searches, pulls fresh reviews live, drops non-English content, and scores with both required models. The headline UI elements are the "players liked/disliked" generated summary and a curated sample of the longest substantive reviews on each side of the vote — the ones with enough detail to actually inform a buying decision — shown alongside the before/after sentiment score (led by Steam's own category label, e.g. "Mostly Positive", rather than the raw percentage) and model-vs-Steam-vote comparison. Per-request inference (~65ms for a 1,000-review batch) is negligible next to the multi-second Steam API call the app already makes; the summarizer only ever runs on the ~6 already-filtered reviews shown in the UI, not the full batch. No GPU is needed in production — the CPU build of TensorFlow keeps the deployed footprint small enough for a free-tier host.

11. Data Answer

Review text alone turns out to carry a generalisable signal for both quality and sentiment, and that signal held up across genuinely different genres, not just the ones each model was tuned on. Confidence is highest for

the quality filter, where precision sits in a tight 0.90–0.96 band across all seven games, including Cyberpunk 2077's open-world RPG reviews — the style least represented in the dataset. Sentiment is a moderate-to-high confidence result: the contest between model families was closer here, but the deployed model's agreement with Steam's real vote stayed consistently ahead of the alternatives across games.

12. Business Answer

Yes — the quality filter can reliably separate substantive reviews from thin ones, letting the app surface real detail (gameplay, bugs, pacing, value) instead of a raw, unsorted list. The resulting "real sentiment" score is a useful secondary check: it reads consistently more critically than Steam's raw score once low-effort noise is removed. That comes with a caveat worth stating plainly: the filter rests on a proxy label for "low-effort" rather than verified ground truth, and the sample reflects recent reviews rather than a game's full history. Read that way, this is a tool for finding informative reviews and getting a more honest sentiment snapshot — not a wholesale replacement for Steam's own aggregate.

13. Response to Stakeholders

- Players: read the surfaced substantive reviews first for actual buying-decision detail, and use the filtered score alongside Steam's own as a secondary check — especially useful right after a patch, price change, or controversy, when low-effort review volume spikes and the raw score is most likely to be misleading.
- Developers/publishers: the surfaced substantive reviews and the filtered score give a clearer read on genuine feedback than the raw aggregate, without needing to read thousands of reviews manually.
- Recommendation: treat this as a decision-support signal, not a single ground-truth number — the underlying label is a proxy, and results are scoped to English-language reviews.

14. End-to-End Solution

UI: Streamlit web app (game name or appid search, curated sample of substantive reviews on each side of the vote, before/after sentiment comparison, flagged-review sample for transparency). Model: CNN (quality) + Stacking ensemble (sentiment), both trained via a reproducible src/train_models.py pipeline separated from the exploratory notebook. Data: Steam's public appreviews API, pulled live for any game, no re-training needed for new titles. Infrastructure: GitHub for source control and CI-free redeploy-on-push; Streamlit Community Cloud for hosting (free tier, sleeps when idle). Tools/libraries: Python, pandas, scikit-learn, TensorFlow/Keras, Streamlit, requests.

15. Limitations & Next Steps

- The "low-effort" label is a proxy, not verified ground truth — a review can be short but insightful, or long but empty.
- English-only scope: results reflect English-speaking reviewers specifically, not a game's full community.
- A most-recent-reviews sample, not full history — can diverge from a game's all-time score (Helldivers 2, Section 7).
- The "Players liked/disliked" summary is generated text (Falconsai/text_summarization, pulled from the ~6 longest kept reviews per game, not run across the full pulled batch), not extracted phrases — local testing across multiple runs showed no outright hallucination, but as generated prose it carries a weaker groundedness guarantee than literal extraction would. Each review is summarized individually and the results joined, so the output reads as several short sentences placed back-to-back rather than one fully unified paragraph — a deliberate trade-off to guarantee every review is represented.
- Next: validate against a broader genre spread; calibrate the flagging threshold against a specific precision/recall target; extend to other languages; evaluate a pretrained transformer for sentiment to catch sarcasm and mixed-signal reviews.
- Next (summarization): a summarizer that synthesizes across multiple reviews in one coherent pass, rather than joining independently-generated per-review summaries — the per-review approach was chosen deliberately to guarantee no review gets dropped, at some cost to how naturally the joined result flows. Five summarizer models were tried in total to reach the current approach (t5-small, bart-large-cnn, distilbart-cnn-12-6, flan-t5, and an interim extractive TF-IDF approach) before Falconsai/text_summarization was adopted.
- Next (scope): extend filtering and summarization to the full pulled batch (up to 1,000 reviews), not just the curated ~6-review sample shown today. This was the original goal behind the summarization feature — giving a picture of what players broadly say, not just a handful of the longest reviews — and closing that gap is the clearest way to fully meet it.

16. References

Code and notebook: github.com/brighthikaru/steam-review-filter-and-score. Live app: steam-review-filter-and-score.streamlit.app. Data source: Steam appreviews API. Libraries: scikit-learn, TensorFlow/Keras, pandas, Streamlit, requests.